

Operating system

An operating system (OS) is system software that manages computer hardware and software resources, and provides common services for computer programs. Time-sharing operating systems schedule tasks for efficient use of the system and may also include accounting software for cost allocation of processor time, mass storage, peripherals, and other resources. For hardware functions such as input and output and memory allocation, the operating system acts as an intermediary between programs and the computer hardware, although the application code is usually executed directly by the hardware and frequently makes system calls to an OS function or is interrupted by it. Operating systems are found on many devices that contain a computer – from cellular phones and video game consoles to web servers and supercomputers. As of November 2025, Android is the most popular operating system, with a 38% market share, followed by Microsoft Windows at 33%, iOS and iPadOS at 15%, macOS at 4%, and Linux at 1%. Android, iOS, and iPadOS are operating systems for mobile devices such as smartphones, while Windows, macOS, and Linux are for desktop computers. Linux distributions are dominant in the server and supercomputing sectors and number in the thousands. Other specialized classes of operating systems (special-purpose operating systems), such as embedded and real-time systems, exist for many applications. Security-focused operating systems also exist. Some operating systems have low system requirements (e.g. light-weight Linux distribution). Others may have higher system requirements. Some operating systems require installation or may come pre-installed with purchased computers (OEM-installation), whereas others may run directly from media (i.e. live CD) or flash memory (i.e. a LiveUSB from a USB stick).

Definition and purpose An operating system is difficult to define, but has been called "the layer of software that manages a computer's resources for its users and their applications". Operating systems include the software that is always running, called a kernel—but can include other software as well. The two other types of programs that can run on a computer are system programs—which are associated with the operating system, but may not be part of the kernel—and applications—all other software. There are three main purposes that an operating system fulfills:

Operating systems allocate resources between different applications, deciding when they will receive central processing unit (CPU) time or space in memory. On modern personal computers, users often want to run several applications at once. In order to ensure that one program cannot monopolize the computer's limited hardware resources, the operating system gives each application a share of the resource, either in time (CPU) or space (memory). The operating system also must isolate applications from each other to protect them from errors and security vulnerabilities in another application's code, but enable communications between different applications. Operating systems provide an interface that abstracts the details of hardware access (such as physical memory) to make things easier for programmers. Virtualization also enables the operating system to mask limited hardware resources; for example, virtual memory can provide a program with the illusion of nearly unlimited memory that exceeds the computer's actual memory. Operating systems provide common services, such as an interface for accessing network and disk devices. This enables an application to be run on different hardware without needing to be rewritten. Which services to include in an operating system varies greatly, and this functionality makes up the great majority of code for most operating systems.

Types of operating systems

Multicomputer operating systems With multiprocessors multiple CPUs share memory. A multicomputer or cluster computer has multiple CPUs, each of which has its own memory. Multicomputers were developed because large multiprocessors are difficult to engineer and prohibitively expensive; they are universal in cloud computing because of the size of the

machine needed. The different CPUs often need to send and receive messages to each other; to ensure good performance, the operating systems for these machines need to minimize this copying of packets. Newer systems are often multiqueue—separating groups of users into separate queues—to reduce the need for packet copying and support more concurrent users. Another technique is remote direct memory access, which enables each CPU to access memory belonging to other CPUs. Multicomputer operating systems often support remote procedure calls where a CPU can call a procedure on another CPU, or distributed shared memory, in which the operating system uses virtualization to generate shared memory that does not physically exist.

Distributed systems A distributed system is a group of distinct, networked computers—each of which might have their own operating system and file system. Unlike multicomputers, they may be dispersed anywhere in the world. Middleware, an additional software layer between the operating system and applications, is often used to improve consistency. Although it functions similarly to an operating system, it is not a true operating system.

Embedded Embedded operating systems are designed to be used in embedded computer systems whether they are internet of things objects or not connected to a network. Embedded systems include many household appliances. The distinguishing factor is that they do not load user-installed software. Consequently, they do not need protection between different applications, enabling simpler designs. Very small operating systems might run in less than 10 kilobytes, and the smallest are for smart cards. Examples include Embedded Linux, QNX, VxWorks, and the extra-small systems RIOT and TinyOS.

Real-time A real-time operating system is an operating system that guarantees to process events or data by or at a specific moment in time. Hard real-time systems require exact timing and are common in manufacturing, avionics, military, and other similar uses. With soft real-time systems, the occasional missed event is acceptable; this category often includes audio or multimedia systems, as well as smartphones. In order for hard real-time systems be sufficiently exact in their timing, often they are just a library with no protection between applications, such as eCos.

Hypervisor A hypervisor is an operating system that runs a virtual machine. The virtual machine is an application that emulates hardware; in other words, it operates as much as possible like the actual hardware the operating system was designed to run on. Virtual machines can be paused, saved, and resumed, making them useful for operating systems research, development, and debugging. They also enhance portability by enabling applications to be run on a computer even if they are not compatible with the base operating system.

Library A library operating system (libOS) is one in which the services that a typical operating system provides, such as networking, are provided in the form of libraries and composed with a single application and configuration code to construct a unikernel: a specialized (only the absolute necessary pieces of code are extracted from libraries and bound together), single address space, machine image that can be deployed to cloud or embedded environments. The operating system code and application code are not executed in separated protection domains (there is only a single application running, at least conceptually, so there is no need to prevent interference between applications) and OS services are accessed via simple library calls (potentially inlining them based on compiler thresholds), without the usual overhead of context switches, in a way similarly to embedded and real-time OSes. This overhead is not negligible: to the direct cost of mode switching it's necessary to add the indirect pollution of important processor structures (like CPU caches, the instruction pipeline, and so on) which affects both user-mode and kernel-mode performance.

History

The first computers in the late 1940s and 1950s were directly programmed either with

plugboards or with machine code inputted on media such as punch cards, without programming languages or operating systems. After the introduction of the transistor in the mid-1950s, mainframes began to be built. These still needed professional operators who manually do what a modern operating system would do, such as scheduling programs to run, but mainframes still had rudimentary operating systems such as Fortran Monitor System (FMS) and IBSYS. In the 1960s, IBM introduced the first series of intercompatible computers (System/360). All of them ran the same operating system—OS/360—which consisted of millions of lines of assembly language that had thousands of bugs. The OS/360 also was the first popular operating system to support multiprogramming, such that, if one job is blocked waiting for an input/output (I/O) operation to complete, another job can use the CPU. Holding multiple jobs in memory required memory partitioning and safeguards against one job accessing the memory allocated to a different job. Around the same time, teleprinters began to be used as terminals so multiple users could access the computer simultaneously. The operating system MULTICS was intended to allow hundreds of users to access a large computer. Despite its limited adoption, it can be considered the precursor to cloud computing. The UNIX operating system originated as a development of MULTICS for a single user. Because UNIX's source code was available, it became the basis of other, incompatible operating systems, of which the most successful were AT&T's System V and the University of California's Berkeley Software Distribution (BSD). To increase compatibility, the IEEE released the POSIX standard for operating system application programming interfaces (APIs), which is supported by most UNIX systems. MINIX was a stripped-down version of UNIX, developed in 1987 for educational uses, that inspired the commercially available, free software Linux. Since 2008, MINIX is used in controllers of most Intel microchips, while Linux is widespread in data centers and Android smartphones.

Microcomputers

The invention of large scale integration enabled the production of personal computers (initially called microcomputers) from around 1980. For around five years, CP/M (Control Program for Microcomputers) was the most popular operating system for microcomputers. Later, IBM bought a disk operating system from Microsoft, which IBM sold as IBM PC DOS and Microsoft branded as MS-DOS (MicroSoft Disk Operating System) and was widely used on IBM PC compatible microcomputers. Later versions increased their sophistication, in part by borrowing features from UNIX. The first popular computer to use a graphical user interface (GUI) was Apple's Macintosh. The GUI proved much more user friendly than the text-only command-line interface of earlier operating systems. This success prompted Microsoft to add a GUI overlay to MS-DOS called Windows. Windows later was rewritten as a stand-alone operating system, Windows NT, borrowing so many features from another OS (VAX/VMS) that a large legal settlement was paid. In the 21st century, Windows continues to be popular on personal computers but has less market share of servers. UNIX operating systems, especially Linux, are the most popular on enterprise systems and servers but are also used on mobile devices and many other computer systems. On mobile devices, Symbian OS was dominant at first, being usurped by BlackBerry OS (introduced 2002) and iOS for iPhones (from 2007). Later on, the open-source Android operating system (introduced 2008), with a Linux kernel and a C library (Bionic) partially based on BSD code, became most popular.

Components The components of an operating system are designed to ensure that various parts of a computer function cohesively. With the de facto obsolescence of DOS, all user software must interact with the operating system to access hardware.

Kernel

The kernel is the part of the operating system that provides protection between different applications and users. This protection is key to improving reliability by keeping errors isolated to one program, as well as security by limiting the power of malicious software and protecting private data, and ensuring that one program cannot monopolize the computer's

resources. Most operating systems have two modes of operation: in user mode, the hardware checks that the software is only executing legal instructions, whereas the kernel has unrestricted powers and is not subject to these checks. The kernel also manages memory for other processes and controls access to input/output devices.

Program execution

The operating system provides an interface between an application program and the computer hardware, so that an application program can interact with the hardware only by obeying rules and procedures programmed into the operating system. The operating system is also a set of services which simplify development and execution of application programs. Executing an application program typically involves the creation of a process by the operating system kernel, which assigns memory space and other resources, establishes a priority for the process in multi-tasking systems, loads program binary code into memory, and initiates execution of the application program, which then interacts with the user and with hardware devices. However, in some systems an application can request that the operating system execute another application within the same process, either as a subroutine or in a separate thread, e.g., the LINK and ATTACH facilities of OS/360 and successors.

Interrupts

An interrupt (also known as an abort, exception, fault, signal, or trap) provides an efficient way for most operating systems to react to the environment. Interrupts cause the central processing unit (CPU) to have a control flow change away from the currently running program to an interrupt handler, also known as an interrupt service routine (ISR). An interrupt service routine may cause the central processing unit (CPU) to have a context switch. The details of how a computer processes an interrupt vary from architecture to architecture, and the details of how interrupt service routines behave vary from operating system to operating system. However, several interrupt functions are common. The architecture and operating system must:

transfer control to an interrupt service routine. save the state of the currently running process. restore the state after the interrupt is serviced.

Software interrupt A software interrupt is a message to a process that an event has occurred. This contrasts with a hardware interrupt—which is a message to the central processing unit (CPU) that an event has occurred. Software interrupts are similar to hardware interrupts—there is a change away from the currently running process. Similarly, both hardware and software interrupts execute an interrupt service routine. Software interrupts may be normally occurring events. It is expected that a time slice will occur, so the kernel will have to perform a context switch. A computer program may set a timer to go off after a few seconds in case too much data causes an algorithm to take too long. Software interrupts may be error conditions, such as a malformed machine instruction. However, the most common error conditions are division by zero and accessing an invalid memory address. Users can send messages to the kernel to modify the behavior of a currently running process. For example, in the command-line environment, pressing the interrupt character (usually Control-C) might terminate the currently running process. To generate software interrupts for x86 CPUs, the INT assembly language instruction is available. The syntax is INT X, where X is the offset number (in hexadecimal format) to the interrupt vector table.

Signal To generate software interrupts in Unix-like operating systems, the kill(pid,signal) system call will send a signal to another process. pid is the process identifier of the receiving process. signal is the signal number (in mnemonic format) to be sent. (The abrasive name of kill was chosen because early implementations only terminated the process.) In Unix-like operating systems, signals inform processes of the occurrence of asynchronous events. To communicate asynchronously, interrupts are required. One reason a process needs to asynchronously communicate to another process is to solve a variation of the classic

reader/writer problem. The writer receives a pipe from the shell for its output to be sent to the reader's input stream. The command-line syntax is alpha | bravo. alpha will write to the pipe when its computation is ready and then sleep in the wait queue. bravo will then be moved to the ready queue and soon will read from its input stream. The kernel will generate software interrupts to coordinate the piping. Signals may be classified into 7 categories. The categories are:

when a process finishes normally. when a process has an error exception. when a process runs out of a system resource. when a process executes an illegal instruction. when a process sets an alarm event. when a process is aborted from the keyboard. when a process has a tracing alert for debugging.

Hardware interrupt Input/output (I/O) devices are slower than the CPU. Therefore, it would slow down the computer if the CPU had to wait for each I/O to finish. Instead, a computer may implement interrupts for I/O completion, avoiding the need for polling or busy waiting. Some computers require an interrupt for each character or word, costing a significant amount of CPU time. Direct memory access (DMA) is an architecture feature to allow devices to bypass the CPU and access main memory directly. (Separate from the architecture, a device may perform direct memory access to and from main memory either directly or via a bus.)

Input/output

Drivers

The operating system includes device drivers to access input/output devices.

Interrupt-driven I/O

When a computer user types a key on the keyboard, typically the character appears immediately on the screen. Likewise, when a user moves a mouse, the cursor immediately moves across the screen. Each keystroke and mouse movement generates an interrupt called Interrupt-driven I/O. An interrupt-driven I/O occurs when a process causes an interrupt for every character or word transmitted.

Direct memory access Devices such as hard disk drives, solid-state drives, and magnetic tape drives can transfer data at a rate high enough that interrupting the CPU for every byte or word transferred, and having the CPU transfer the byte or word between the device and memory, would require too much CPU time. Data is, instead, transferred between the device and memory independently of the CPU by hardware such as a channel or a direct memory access controller; an interrupt is delivered only when all the data is transferred. If a computer program executes a system call to perform a block I/O write operation, then the system call might execute the following instructions:

Set the contents of the CPU's registers (including the program counter) into the process control block. Create an entry in the device-status table. The operating system maintains this table to keep track of which processes are waiting for which devices. One field in the table is the memory address of the process control block. Place all the characters to be sent to the device into a memory buffer. Set the memory address of the memory buffer to a predetermined device register. Set the buffer size (an integer) to another predetermined register. Execute the machine instruction to begin the writing. Perform a context switch to the next process in the ready queue. While the writing takes place, the operating system will context switch to other processes as normal. When the device finishes writing, the device will interrupt the currently running process by asserting an interrupt request. The device will also place an integer onto the data bus. Upon accepting the interrupt request, the operating system will:

Push the contents of the program counter (a register) followed by the status register onto the call stack. Push the contents of the other registers onto the call stack.

(Alternatively, the contents of the registers may be placed in a system table.) Read the integer from the data bus. The integer is an offset to the interrupt vector table. The vector table's instructions will then: Access the device-status table. Extract the process control block. Perform a context switch back to the writing process. When the writing process has its time slice expired, the operating system will:

Pop from the call stack the registers other than the status register and program counter. Pop from the call stack the status register. Pop from the call stack the address of the next instruction, and set it back into the program counter. With the program counter now reset, the interrupted process will resume its time slice.

Memory management

Among other things, a multiprogramming operating system kernel must be responsible for managing all system memory which is currently in use by the programs. This ensures that a program does not interfere with memory already in use by another program. Since programs time share, each program must have independent access to memory. Cooperative memory management, used by many early operating systems, assumes that all programs make voluntary use of the kernel's memory manager, and do not exceed their allocated memory. This system of memory management is almost never seen anymore, since programs often contain bugs which can cause them to exceed their allocated memory. If a program fails, it may cause memory used by one or more other programs to be affected or overwritten. Malicious programs or viruses may purposefully alter another program's memory, or may affect the operation of the operating system itself. With cooperative memory management, it takes only one misbehaved program to crash the system. Memory protection enables the kernel to limit a process' access to the computer's memory. Various methods of memory protection exist, including memory segmentation and paging. All methods require some level of hardware support (such as the 80286 MMU), which does not exist in all computers. In both segmentation and paging, certain protected mode registers specify to the CPU what memory address it should allow a running program to access. Attempts to access other addresses trigger an interrupt, which causes the CPU to re-enter supervisor mode, placing the kernel in charge. This is called a segmentation violation or Seg-V for short, and since it is both difficult to assign a meaningful result to such an operation, and because it is usually a sign of a misbehaving program, the kernel generally resorts to terminating the offending program, and reports the error. Windows versions 3.1 through ME had some level of memory protection, but programs could easily circumvent the need to use it. A general protection fault would be produced, indicating a segmentation violation had occurred; however, the system would often crash anyway.

Virtual memory

The use of virtual memory addressing (such as paging or segmentation) means that the kernel can choose what memory each program may use at any given time, allowing the operating system to use the same memory locations for multiple tasks. If a program tries to access memory that is not accessible memory, but nonetheless has been allocated to it, the kernel is interrupted (see § Memory management). This kind of interrupt is typically a page fault. When the kernel detects a page fault it generally adjusts the virtual memory range of the program which triggered it, granting it access to the memory requested. This gives the kernel discretionary power over where a particular application's memory is stored, or even whether or not it has been allocated yet. In modern operating systems, memory which is accessed less frequently can be temporarily stored on a disk or other media to make that space available for use by other programs. This is called swapping, as an area of memory can be used by multiple programs, and what that memory area contains can be swapped or exchanged on demand. Virtual memory provides the programmer or the user with the perception that there is a much larger amount of RAM in the computer than is really there.

Concurrency

Concurrency refers to the operating system's ability to carry out multiple tasks simultaneously. Virtually all modern operating systems support concurrency. Threads enable splitting a process' work into multiple parts that can run simultaneously. The number of threads is not limited by the number of processors available. If there are more threads than processors, the operating system kernel schedules, suspends, and resumes threads, controlling when each thread runs and how much CPU time it receives. During a context switch a running thread is suspended, its state is saved into the thread control block and stack, and the state of the new thread is loaded in. Historically, on many systems a thread could run until it relinquished control (cooperative multitasking). Because this model can allow a single thread to monopolize the processor, most operating systems now can interrupt a thread (preemptive multitasking). Threads have their own thread ID, program counter (PC), a register set, and a stack, but share code, heap data, and other resources with other threads of the same process. Thus, there is less overhead to create a thread than a new process. On single-CPU systems, concurrency is switching between processes. Many computers have multiple CPUs. Parallelism with multiple threads running on different CPUs can speed up a program, depending on how much of it can be executed concurrently.

File system

Permanent storage devices used in twenty-first century computers, unlike volatile dynamic random-access memory (DRAM), are still accessible after a crash or power failure. Permanent (non-volatile) storage is much cheaper per byte, but takes several orders of magnitude longer to access, read, and write. The two main technologies are a hard drive consisting of magnetic disks, and flash memory (a solid-state drive that stores data in electrical circuits). The latter is more expensive but faster and more durable. File systems are an abstraction used by the operating system to simplify access to permanent storage. They provide human-readable filenames and other metadata, increase performance via amortization of accesses, prevent multiple threads from accessing the same section of memory, and include checksums to identify corruption. File systems are composed of files (named collections of data, of an arbitrary size) and directories (also called folders) that list human-readable filenames and other directories. An absolute file path begins at the root directory and lists subdirectories divided by punctuation, while a relative path defines the location of a file from a directory. System calls (which are sometimes wrapped by libraries) enable applications to create, delete, open, and close files, as well as link, read, and write to them. All these operations are carried out by the operating system on behalf of the application. The operating system's efforts to reduce latency include storing recently requested blocks of memory in a cache and prefetching data that the application has not asked for, but might need next. Device drivers are software specific to each input/output (I/O) device that enables the operating system to work without modification over different hardware. Another component of file systems is a dictionary that maps a file's name and metadata to the data block where its contents are stored. Most file systems use directories to convert file names to file numbers. To find the block number, the operating system uses an index (often implemented as a tree). Separately, there is a free space map to track free blocks, commonly implemented as a bitmap. Although any free block can be used to store a new file, many operating systems try to group together files in the same directory to maximize performance, or periodically reorganize files to reduce fragmentation. Maintaining data reliability in the face of a computer crash or hardware failure is another concern. File writing protocols are designed with atomic operations so as not to leave permanent storage in a partially written, inconsistent state in the event of a crash at any point during writing. Data corruption is addressed by redundant storage (for example, RAID—redundant array of inexpensive disks) and checksums to detect when data has been corrupted. With multiple layers of checksums and backups of a file, a system can recover from multiple hardware failures. Background processes are often used to detect and recover from data corruption.

Networking

Modern operating systems usually include a network stack, such as the TCP/IP protocol stack.

Security

Security means protecting users from other users of the same computer, as well as from those who seeking remote access to it over a network. Operating systems security rests on achieving the CIA triad: confidentiality (unauthorized users cannot access data), integrity (unauthorized users cannot modify data), and availability (ensuring that the system remains available to authorized users, even in the event of a denial of service attack). As with other computer systems, isolating security domains—in the case of operating systems, the kernel, processes, and virtual machines—is key to achieving security. Other ways to increase security include simplicity to minimize the attack surface, locking access to resources by default, checking all requests for authorization, principle of least authority (granting the minimum privilege essential for performing a task), privilege separation, and reducing shared data. Some operating system designs are more secure than others. Those with no isolation between the kernel and applications are least secure, while those with a monolithic kernel like most general-purpose operating systems are still vulnerable if any part of the kernel is compromised. A more secure design features microkernels that separate the kernel's privileges into many separate security domains and reduce the consequences of a single kernel breach. Unikernels are another approach that improves security by minimizing the kernel and separating out other operating systems functionality by application. Most operating systems are written in C or C++, which create potential vulnerabilities for exploitation. Despite attempts to protect against them, vulnerabilities are caused by buffer overflow attacks, which are enabled by the lack of bounds checking. Hardware vulnerabilities, some of them caused by CPU optimizations, can also be used to compromise the operating system. There are known instances of operating system programmers deliberately implanting vulnerabilities, such as back doors. Operating systems security is hampered by their increasing complexity and the resulting inevitability of bugs. Because formal verification of operating systems may not be feasible, developers use operating system hardening to reduce vulnerabilities, e.g. address space layout randomization, control-flow integrity, access restrictions, and other techniques. There are no restrictions on who can contribute code to open source operating systems; such operating systems have transparent change histories and distributed governance structures. Open source developers strive to work collaboratively to find and eliminate security vulnerabilities, using code review and type checking to expunge malicious code. Andrew S. Tanenbaum advises releasing the source code of all operating systems, arguing that it prevents developers from placing trust in secrecy and thus relying on the unreliable practice of security by obscurity.

User interface

A user interface (UI) is essential to support human interaction with a computer. The two most common user interface types for any computer are

command-line interface, where computer commands are typed, line-by-line, graphical user interface (GUI) using a visual environment, most commonly a combination of the window, icon, menu, and pointer elements, also known as WIMP. For personal computers, including smartphones and tablet computers, and for workstations, user input is typically from a combination of keyboard, mouse, and trackpad or touchscreen, all of which are connected to the operating system with specialized software. Personal computer users who are not software developers or coders often prefer GUIs for both input and output; GUIs are supported by most personal computers. The software to support GUIs is more complex than a command line for input and plain text output. Plain text output is often preferred by programmers, and is easy to support.

Operating system development as a hobby

A hobby operating system may be classified as one whose code has not been directly derived from an existing operating system, and has few users and active developers. In some cases, hobby development is in support of a "homebrew" computing device, for example, a simple single-board computer powered by a 6502 microprocessor. Or, development may be for an architecture already in widespread use. Operating system development may come from entirely new concepts, or may commence by modeling an existing operating system. In either case, the hobbyist is her/his own developer, or may interact with a small and sometimes unstructured group of individuals who have like interests. Examples of hobby operating systems include Syllable and TempleOS.

Diversity of operating systems and portability If an application is written for use on a specific operating system, and is ported to another OS, the functionality required by that application may be implemented differently by that OS (the names of functions, meaning of arguments, etc.) requiring the application to be adapted, changed, or otherwise maintained. This cost in supporting operating systems diversity can be avoided by instead writing applications for software platforms such as Java or Qt. These abstractions have already borne the cost of adaptation to specific operating systems and their system libraries. Another approach is for operating system vendors to adopt standards. For example, POSIX and OS abstraction layers provide commonalities that reduce porting costs.

Popular operating systems

As of October 2025, metrics for web traffic show that Android is the most popular operating system used to connect to the web, with a 38% share, followed by Microsoft Windows at 31%, iOS and iPadOS at 15%, macOS at 7%, and Linux at 1%. Android, iOS, and iPadOS are mobile operating systems, while Windows, macOS, and Linux are desktop operating systems.

Linux

Linux is free software distributed under the GNU General Public License (GPL), which means that all of its derivatives are legally required to release their source code. Linux was designed by programmers for their own use, thus emphasizing simplicity and consistency, with a small number of basic elements that can be combined in nearly unlimited ways, and avoiding redundancy. Its design is similar to other UNIX systems not using a microkernel. It is written in C and uses UNIX System V syntax, but also supports BSD syntax. Linux supports standard UNIX networking features, as well as the full suite of UNIX tools, while supporting multiple users and employing preemptive multitasking. Initially of a minimalist design, Linux is a flexible system that can work in under 16 MB of RAM, but still is used on large multiprocessor systems. Similar to other UNIX systems, Linux distributions are composed of a kernel, system libraries, and system utilities. Linux has a graphical user interface (GUI) with a desktop, folder and file icons, as well as the option to access the operating system via a command line. Android is a partially open-source operating system closely based on Linux and has become the most widely used operating system by users, due to its popularity on smartphones and, to a lesser extent, embedded systems needing a GUI, such as "smart watches, automotive dashboards, airplane seatbacks, medical devices, and home appliances". Unlike Linux, much of Android is written in Java and uses object-oriented design.

Microsoft Windows

Windows is a proprietary operating system that is widely used on desktop computers, laptops, tablets, phones, workstations, enterprise servers, and Xbox consoles. The operating system was designed for "security, reliability, compatibility, high performance, extensibility, portability, and international support"—later on, energy efficiency and support for dynamic devices also became priorities. Windows Executive works via kernel-mode objects for important data structures like processes, threads, and sections (memory objects, for example files). The operating system supports demand paging of virtual memory, which speeds up I/O for many applications. I/O device drivers use the Windows Driver Model. The NTFS file system

has a master table and each file is represented as a record with metadata. The scheduling includes preemptive multitasking. Windows has many security features; especially important are the use of access-control lists and integrity levels. Every process has an authentication token and each object is given a security descriptor. Later releases have added even more security features.

See also

Notes

References

Sources

Further reading

External links

Multics History and the history of operating systems